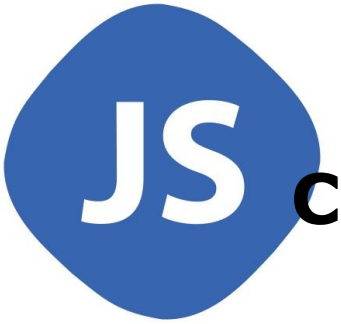


Callback функции, часть 1

- Введение
- API интерфейсы
- Что такое callback
- Создаем callback
- Варианты callback

Введение

В своей основной форме **JavaScript** является **синхронным**, **блокирующим**, **однопоточным** языком, в котором одновременно может выполняться только одна операция.

Все потому, что он может выполнять только одну команду за раз и в определенном программистом порядке. Это означает, что выполнение кода будет выполняться по частям. Поскольку выполнение кода выполняется последовательно, любой код, выполнение которого занимает много времени, **блокирует** все, что необходимо выполнить после этого.

Следовательно, если какая-либо другая команда хочет что-то выполнить, она должна дождаться полного выполнения предыдущей команды. Этот поток обычно известен как **основной поток** или **основной поток выполнения**.

В демонстрационном примере **example_1** простой сценарий вывода данных массива в документ браузера.

example_1. Последовательное выполнение кода

```
<script>

    // инициализация массива

    arr = [

        'Твои б мозги да к моему диплому!..',
```

```
        'Чего б еще разумного посеять?',
        'Тефтеля – это вам не фунт изюма!',
        'Контрольный выстрел мало что исправил…',
        'Упал кирпич на голову. К чему бы?',
        'На минус 2 кило я похудела.',
        'Два дня не сплю, не ем уже три ночи...',
        'Вы идиот?! Нет, нет, не отвечайте!..',
        'Когда умру, прошу – без ликованья...'
    ]
    var i = 0; // итератор цикла
    var line = "";
    // проходим по массиву циклом while
    while (i < arr.length) {
        console.log(i);
        line += "<p>" + arr[i] + "</p>";
        i++;
        // i--; // ошибка в коде программы
    };
    // вставка контента в документ
    document.getElementById("app").insertAdjacentHTML("beforeend"
    , line);
    // вывод об успешном выполнении в диалоговое окно
    alert("The task was completed successfully!");
    // вывод об успешном выполнении в консоль
    console.log("The task was completed successfully!");
</script>
```



В коде сценария команды выполняются одна за другой:

- Инициализация массива **arr**;
- Инициализация переменных: **i**, **line**;
- Цикл вывода в консоль и сохранения элементов массива в HTML-строку;
- Добавление созданной строки в тело HTML документа;
- Вывод сообщения об успешном выполнении.

Пока выполняется каждая операция, ничего больше не может произойти — обработка (отображение) документа приостановлено. Так происходит, потому что JavaScript является **однопоточным**. В каждый момент времени может выполняться только одна команда, обрабатываемая в единственном — главном потоке. Все остальные действия **блокируются** до окончания выполнения текущей команды.

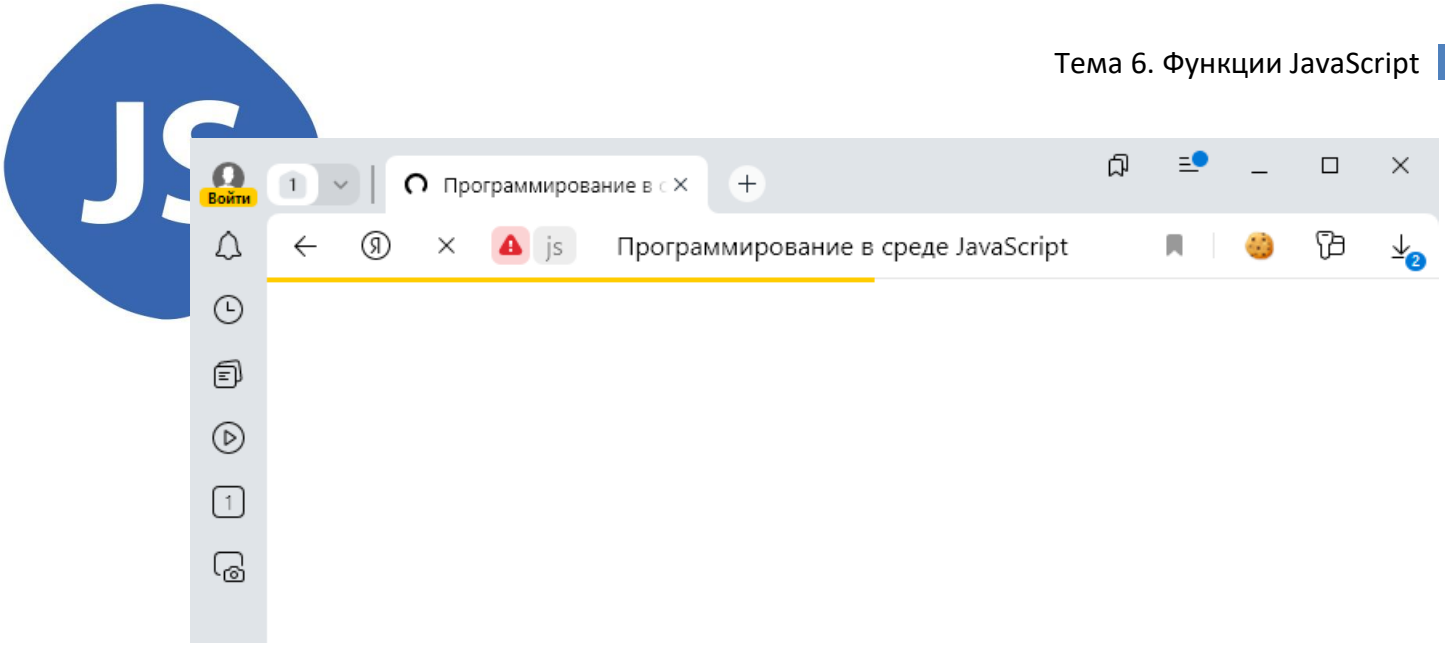
Когда выполнение JavaScript заблокировано, **браузер** просто **зависнет** и не будет ни на что реагировать, пока эта задача не будет завершена.

В примере кода ниже программистом допущена ошибка, что приведет к зависанию браузера в **бесконечном цикле основного потока выполнения**.

```
// проходим по массиву циклом while
while (i < arr.length) {
    console.log(i);
    line += "<p>" + arr[i] + "</p>";
    // i++;
    i--; // ошибка в коде программы
};
```

Любой код после приведенного выше оператора **while** не будет выполняться, поскольку цикл будет повторяться **бесконечно**, пока в системе не закончатся ресурсы.

Зайдите в код примера **example_1** и вызовите данную ошибку.



Но, что делать, если браузер отправляет **HTTP-запрос** для загрузки некоторых данных по сети или загрузки **изображения** для отображения на веб-странице. Зависнет ли браузер до тех пор, пока этот запрос не будет обработан? Если да, то это очень плохо для пользователя.

> ***Примечание***. Ни разработчик, ни браузер не могут знать, как быстро загрузятся данные с **внешнего ресурса**. Пусть это будут текстовые данные или изображение, не важно, загрузка не может быть выполнена мгновенно.

Современные браузеры, не полагаются на один поток JavaScript. Вместо этого они используют **отдельный поток** JavaScript для **каждой вкладки** или для **каждого домена**.

В случае с Google Chrome вы можете открывать несколько вкладок с разными веб-сайтами и **запускать вечный цикл while**. Это только заморозит текущую вкладку, на которой был выполнен этот код, но другие вкладки будут работать нормально. Любая вкладка, на которой открыта страница из того же **домена / того же веб-сайта**, также зависнет, поскольку Google Chrome реализует политику **один процесс на сайт**, а процесс использует тот же поток выполнения JavaScript.

Если бы **браузеру** пришлось использовать один и тот же **поток JavaScript** для выполнения всех задач, пользовательский опыт был бы ужасным.



Например, если браузеру пришлось использовать один и тот же поток JavaScript для выполнения задачи **при получении сетевого ответа HTTP**, тогда веб-страница не отвечала бы в течение нескольких секунд или даже минут.

На самом деле браузер использует больше, чем просто движок JavaScript. **Среда выполнения** JavaScript состоит из еще двух компонентов, а именно **цикл событий** и **очередь обратного вызова**. Очередь обратного вызова также называется **очередью сообщений** или **очередью задач**.

Помимо **движка JavaScript**, браузер содержит приложения, которые могут выполнять различные операции, такие как:

- отправка **HTTP-запросов**;
- прослушивание **событий DOM**;
- задержка выполнения с использованием **setTimeout** или **setInterval**;
- **кеширование**;
- **хранение** данных;
- и многое другое.

Эти функции браузера помогают создавать многофункциональные веб-приложения и улучшать взаимодействие с пользователем. Следовательно, **браузер** реализует свою собственную логику для выполнения задач, таких как отправка HTTP-запросов и прослушивание их ответов.

> ***Важно***. Эти операции **не блокируют основной поток выполнения JavaScript**, поскольку они создаются в разных потоках, **управляемых браузером**, и JavaScript не имеет об этом представления.

Браузер может использовать низкоуровневый язык, такой как C или C++, для реализации задач повышения производительности и **предоставления** нам чистого JavaScript API для выполнения этих операций из JavaScript.

Например, **API fetch** предоставляется браузером для отправки **HTTP-запросов**. На изучении интерфейса **API fetch** построена тема асинхронных



запросов. Эти API-интерфейсы известны как **веб API**, поскольку они не являются частью **спецификаций JavaScript**.

API интерфейсы

Интерфейсы прикладного программирования, или **API** — это способ взаимодействия компьютерных программ или служб друг с другом. Это взаимодействие обычно происходит через конечную точку API, предоставляемую программой, которую использует клиент.

> ***Примечание***. **API** (Application Programming Interface, или интерфейс для программирования приложений) – это механизмы, которые позволяют двум программным компонентам взаимодействовать друг с другом, используя набор определений и протоколов. При этом говорят, что одна из программ предоставляет API-интерфейсы для взаимодействия.

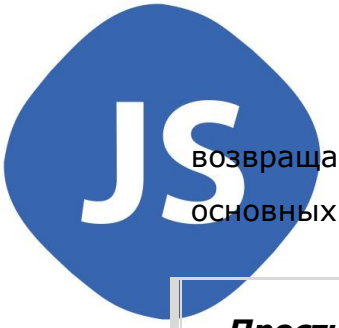
Что такое REST API

REST API — это архитектурный подход, который устанавливает ограничения для API: как они должны быть устроены и какие функции поддерживать. Это позволяет стандартизировать работу программных интерфейсов, сделать их более удобными и производительными.

Слово **REST** — акроним от Representational State Transfer, что переводится на русский как "передача состояния представления", "передача репрезентативного состояния" или "передача самоописываемого состояния".

REST API — список рекомендаций, которым можно следовать или не следовать. Поэтому у него нет собственных методов.

REST API — на сегодняшний день это самые популярные и гибкие API-интерфейсы в Интернете. Клиент отправляет запросы на сервер в виде данных. Сервер использует этот клиентский ввод для запуска внутренних функций и



возвращает выходные данные обратно клиенту. Изучение REST API одна из основных задач курса.

> ***Простыми словами***. **REST API** – это набор правил для организации взаимодействия с сервером.

Что такое Web API

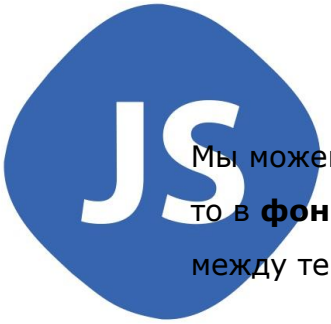
Веб API — это просто интерфейс для доступа к ресурсам сервера по протоколу HTTP. Термин относится к концепции, а не к какой—либо конкретной технологии – веб API может быть создан с использованием различных технологий, таких как Java и ASP.NET. Веб API используют интерфейс с открытым исходным кодом и используют множество клиентских объектов, таких как браузеры, смартфоны, планшеты и ноутбуки.

Веб API может быть REST API, а может и не быть, в зависимости от того, как он построен. Веб API обычно используются в распределенной системе для предоставления услуг на различных устройствах, таких как смартфоны и ноутбуки, и ограничены клиентской частью веб-приложения.

> ***Простыми словами***. **Веб API** – набор методов / функций (одним словом - интерфейс) для доступа к ресурсам сервера по протоколу HTTP.
>
> ***Таким образом***:
>
> - **Веб API** – способ взаимодействия с сервером.
> - **REST API** – правила взаимодействия с сервером.

Цель курса – организация доступа к ресурсам сервера по правилам **REST API**.

Что из этого



Мы можем написать программные инструкции для API-интерфейсов делать что-то в **фоновом режиме и возвращать данные, когда это будет сделано**, а между тем **основной поток выполнения JavaScript будет выполняться**.

Поручая этим API-интерфейсам делать что-то в фоновом режиме, мы должны предоставить **функцию обратного вызова**.

> ***Важно***. **Функция обратного вызова отвечает за выполнение некоторого кода JavaScript в основном потоке Javascript после того, как веб API завершит свою работу**.

Цель урока - разобраться, что такое функции обратного вызова, и как веб разработчик может использовать предоставляемые возможности браузера.

Что такое callback

Callback (**коллбэк**) — это функция, которая должна быть выполнена **после того, как другая функция завершила выполнение** (отсюда название: callback — функция обратного вызова).

Если чуть сложнее, в JavaScript функции — это объекты. Поэтому функции могут **принимать другие функции в качестве аргументов**, а также **возвращать функции в качестве результата** (как любые объекты). Функции, которые это умеют, называются функциями высшего порядка. А любая функция, которая передается как аргумент, называется **callback-функцией**.

> ***Еще раз***. Функция, передаваемая в качестве аргумента другой функции, называется **функцией обратного вызова**, или **callback-функцией**.

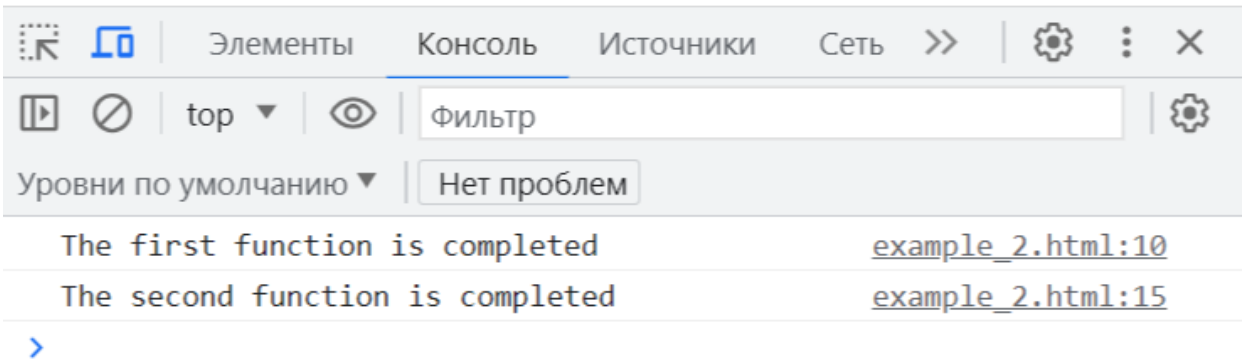
JavaScript — это событийно-ориентированный язык. Поэтому вместо того, чтобы ждать ответа для дальнейшего выполнения программы, JavaScript **продолжит выполнение кода**, одновременно ожидая других событий.

Подойдем к понятию функции обратного вызова на цепочке простых примеров.

example_2. Последовательный вызов двух функций

```
<script>
    // определяем первую функцию
    function first() {
        console.log("The first function is completed");
    }
    // определяем вторую функцию
    function second() {
        console.log("The second function is completed");
    }
    // вызываем функции
    first();
    second();
    // поведение функций предсказуемо
</script>
```

Как мы и ожидали, функция **first** выполнится первой, а функция **second** уже после нее. Поведение функций предсказуемо, в консоли будет выведен следующий результат:





Пока что все понятно. Но что, если функция **first** содержит некий код, который не может выполниться немедленно? К примеру, работа с API, где мы **отправляем запрос к внешнему ресурсу** и должны **ждать ответа**.

Чтобы смоделировать такую ситуацию, мы используем функцию **setTimeout**, которая вызывает пользовательский код (функцию) после заданного временного промежутка.

setTimeout

Синтаксис

```
setTimeout(func|code, [delay], [arg1], [arg2], ...);
```

Параметры

- **func|code –** функция или строка кода для выполнения. Обычно это функция. По историческим причинам можно передать и строку кода, но это не рекомендуется.
- **delay – з**адержка перед запуском в миллисекундах (1000 мс = 1 с). Значение по умолчанию – 0.
- **arg1**, **arg2** – аргументы, передаваемые в функцию

Например, данный код вызывает функцию **sayHi**() спустя одну секунду:

```
// определяем отложенную функцию
function sayHi() {
    alert('Привет!');
}
// запускаем функцию на выполнение
setTimeout(sayHi, 1000);
```

В демонстрационном примере **example_3** происходит вызов функции с передачей аргументов.

example_3. Функция setTimeout с параметрами

```
<script>
    // определяем отложенную функцию
    // функция имеет параметры
    function sayHi(name, login) {
        alert( "Привет, " +  name + " (" + login + ")!" );
    }
    // запускаем функцию на выполнение
    setTimeout(sayHi, 1000, "Евгений", "master");
</script>
```

В следующем примере несколько вариантов вызова с использованием **setTimeout**. Тестируем.

example_4. Тестируем функцию setTimeout

```
<script>
    // определения функций коллбэков
    // первая функция коллбэк
    // функция-инструкция
    function fnFirstCallback() {
        console.log("Первый коллбэк для setTimeout...");
    }
    // вторая функция коллбэк
    // функция-выражение
    fnSecondCallback = function (message) {
        console.log(message);
    }
```

```
    // третья функция коллбэк
    // функция-стрелка
    fnThirdCallback = () => {
        console.log("Третий коллбэк для setTimeout...");
    }
    // вызов функций
    // первая функция коллбэк вызывается через 800 мсек.
    setTimeout(fnFirstCallback, 800);
    // вторая функция коллбэк вызывается через 200 мсек.
    setTimeout(fnSecondCallback, 200, "Второй коллбэк для
    setTimeout...");
    // третья функция коллбэк вызывается через 1100 мсек.
    setTimeout(fnThirdCallback, 1100);
    // четвертая функция коллбэк
    // четвертая функция коллбэк вызывается через 500 мсек.
    setTimeout(() => {console.log("Четвертый коллбэк для
    setTimeout...")}, 500);
</script>
```

В коде демонстрационного примера **example_5**, используем функцию **setTimeout**, которая вызывает стрелочную функцию с отложенным выполнением на 500 миллисекунд, как будто бы это запрос к некому API.

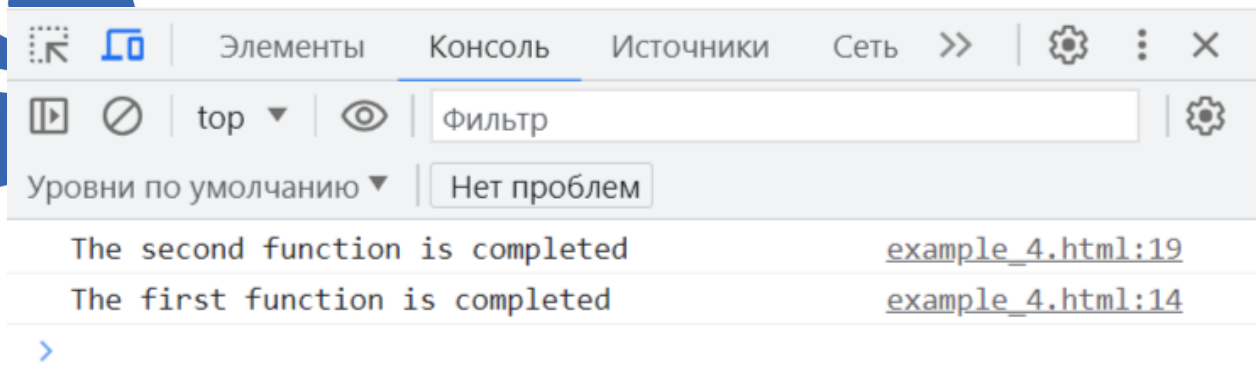
Теперь разбираемый сценарий будет выглядеть следующим образом.

example_5. Моделируем запрос к API

```
<script>
    // в этом скрипте начинаются проблемы
    // определяем первую функцию
```

```
    function first() {
        // моделируем запрос к API (на внешние ресурсы)
        // откладываем выполнение на 500 мсек.
        setTimeout(() => {console.log("The first function is
        completed")}, 500);
    };
    // определяем вторую функцию
    function second() {
        console.log("The second function is completed");
    };
    // вызываем функции
    first();
    second();
    // порядок вызова функций не изменился, но
    // изменился порядок выполнения функций
    // такая ситуация может стать проблемой,
    // если вторая функция должна воспользоваться результатами
    вызова первой
</script>
```

Порядок вызова функций не изменился, но, если посмотреть в консоль, увидим, изменился **порядок их выполнения**. Такая ситуация может стать проблемой при работе с данными.



Даже если вы не понимаете, как работает **setTimeout**(), основная идея в следующем — мы отложили исполнение команды console.log("The first function is completed") на 500 миллисекунд.

Во время выполнения JavaScript находит функцию **first** и исполняет ее с отложенным выводом. Затем, без всяких пауз, он переходит на вторую функцию и тоже запускает ее. JavaScript никогда не останавливается и ищет, что ему еще выполнить (JS - он работоголик, а вы? ;). Именно поэтому в современных браузерах он так быстро работает.

Изменился и изменился. Если мы выводим изображения котиков и собачек в статичный HTML-блок, то можно даже и не подозревать о существующей проблеме. Но такая ситуация недопустима, если **вторая функция** должна воспользоваться **результатами вызова первой**.

Протестируем такую ситуацию на примере получения данных в отложенной функции.

example_6. Моделируем получение внешних данных

```
<script>

    // возвращаемся к проблеме запроса API

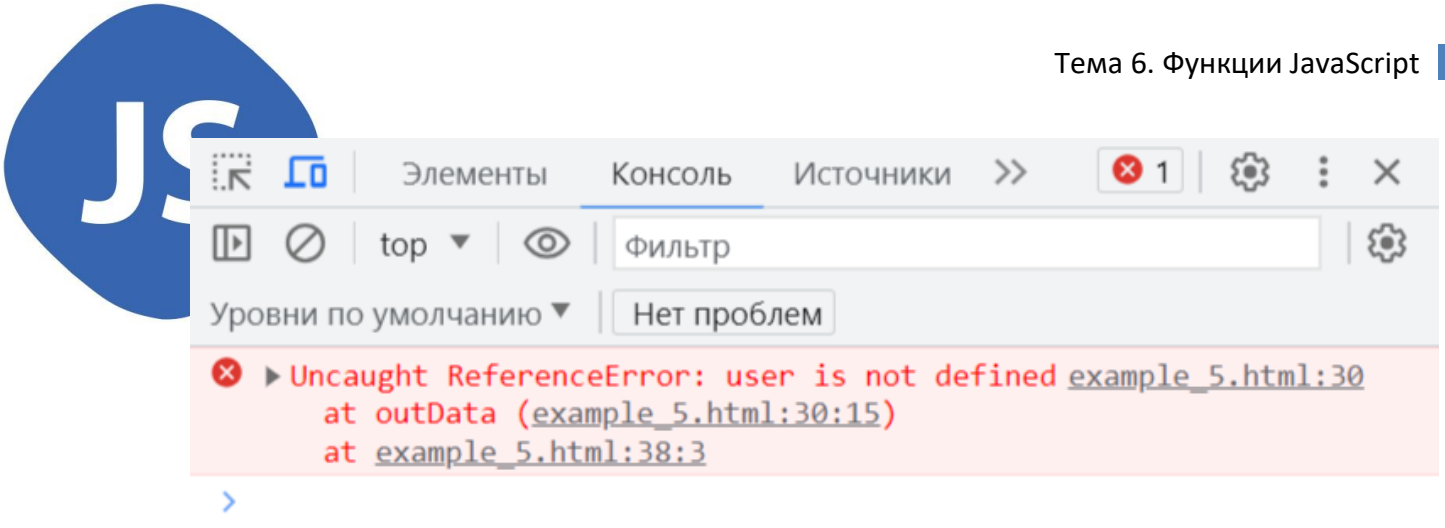
    // вторая функция должна воспользоваться результатами вызова
    первой

    // функция получения данных

    function getData() {
```

```
        // моделируем запрос к API (на внешние ресурсы)
        setTimeout(() => {
            // определение данных
            user = {name:"John", login:"master"}
        }, 500);
    };
    // функция вывода данных
    function  outData() {
        // интерполируем переменные в строку
        html = `
            <h2>Здравствуйте, вы вошли в приложение как:</h2>
            Имя: <b>${user.name}</b> <br>
            Логин: <b>${user.login}</b> <br>
        `;
        // вывод данных в браузер
        document.getElementById("data").innerHTML = html;
    };
    // получаем проблему работы приложения
    getData();
    outData();
</script>
```

Получаем вполне предполагаемую ошибку. Переменная **user** не определена, на момент вызова функции-потребителя **outData**() данные внешнего источника еще не получены.



Хотя мы по-прежнему вызываем функцию получения данных первой, ее вывод появился вторым, после вывода функции **outData**(). Но JavaScript не нарушает порядок вызова функций, он просто **не дожидается ответа** от функции **getData**, а сразу **двигается дальше — к функции outData**.

Попробуем смягчить удар *падаемого* скрипта ;) Предусмотрим ситуацию отсутствия внешних данных.

example_7. Устранение причины остановки скрипта

```
<script>
    // функция получения данных
    function getData() {
        // моделируем запрос к API (на внешние ресурсы)
        setTimeout(() => {
            // определение данных
            user = { name: "John", login: "master" }
        }, 500);
    };
    // функция вывода данных
    function outData() {
        // если данные не определены
        if (user.name == undefined || user.login == undefined)
```

```
            // формируем предупреждение
            html = `
                <h2>Сбой авторизации в системе</h2>
            `
        else
            // иначе интерполируем переменные в строку
            html = `
                <h2>Здравствуйте, вы вошли в приложение как:</h2>
                Имя: <b>${user.name}</b> <br>
                Логин: <b>${user.login}</b> <br>
            `;
        // вывод данных в браузер
        document.getElementById("data").innerHTML = html;
    };
    // инициируем пустой объект
    user = {};
    // проблема решена лишь отчасти
    // внешние данные пришли, но мы их не видим
    getData();
    outData();
</script>
```

Проблема падающего скрипта решена лишь отчасти. Данные внешнего ресурса пришли, но мы их не видим.

Таким образом, нельзя просто вызывать функции в нужном порядке и надеяться, что они в любом случае выполнятся в том же порядке. Коллбэки же позволяют нам быть уверенными в том, что определенный код **не выполнится** до того момента, пока **другой код не завершит своё исполнение**.

Примечание. Изучение темы урока можно продолжать только в том случае, если вы разобрались с причиной появления возможных ошибок получения данных внешних ресурсов.

Создаем callback

Итак, анализ кода демонстрационного примера **example_7**, говорит о том, что вставка данных должна быть выполнена внутри функции, вызываемой стандартной функцией **setTimeout**, т.е. тогда, когда **внешние данные уже получены**.

```
function getData() {
    // моделируем запрос к API (на внешние ресурсы)
    setTimeout(() => {
        // определение данных
        user = {name:"John", login:"master"}
        // в этом месте данные получены
        // можно использовать в сценарии
    }, 500);
};
```

Вставим код вывода полученных данных внутрь функции **setTimeout**().

example_8. Используем данные там же, где их получаем

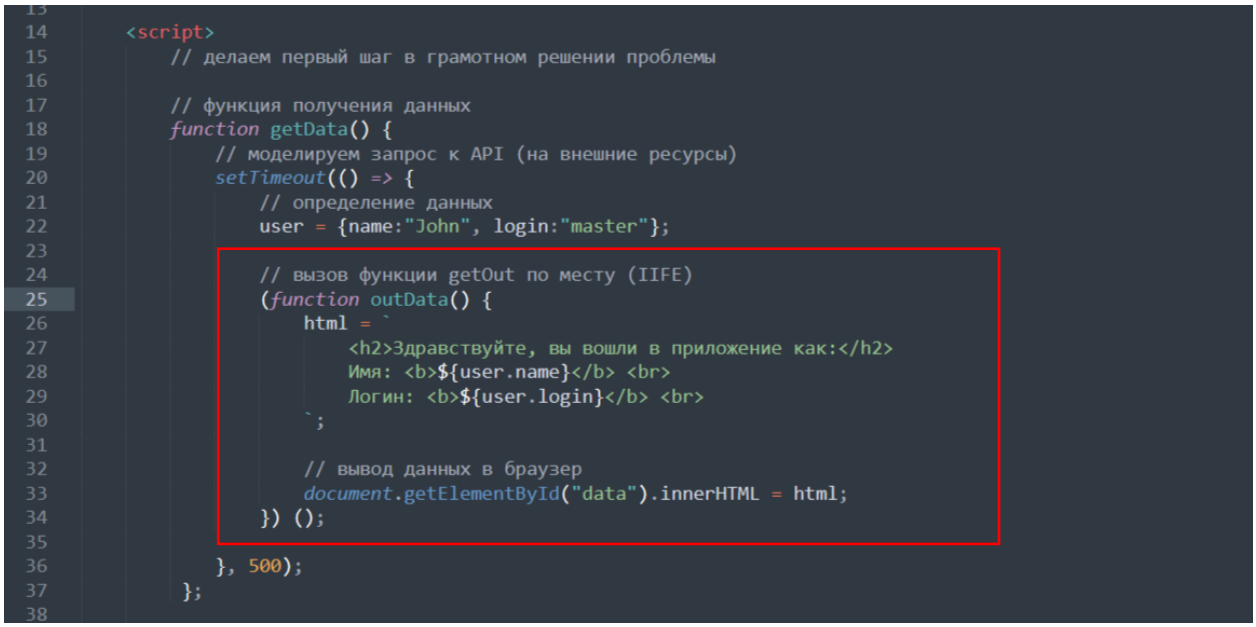
```
<script>
    // проблему решаем тем, что свалили код получения и вывода
    данных в одну кучу
    // функция получения данных
    function getData() {
        // моделируем запрос к API (на внешние ресурсы)
```

```
        setTimeout(() => {
            // определение данных
            user = { name: "John", login: "master" };
            // данные получены, можно использовать в сценарии
            html = `
                <h2>Здравствуйте, вы вошли в приложение как:</h2>
                Имя: <b>${user.name}</b> <br>
                Логин: <b>${user.login}</b> <br>
            `;
            // вывод данных в браузер
            document.getElementById("data").innerHTML = html;
        }, 500);
    };
    // функция вывода данных
    function outData() {
        /*
            а как же я??? :(
        */
    };
    // вызов функций
    getData();
    // outData();
</script>
```

Для создания нашей первой коллбэк функции воспользуюсь знанием **синтаксиса IIFE** (Immediately Invoked Function Expression - немедленно вызываемое функциональное выражение).

Я просто перенес описание функции **outData** внутрь **setTimeOut**, а для ее **немедленного исполнения** обрамил функцию скобками.

Проверяем работу сценария, все прекрасно работает. Сначала определяются данные, потом вызывается функция-потребитель этих данных.



example_9. Код вывода данных оформляем в виде функции

```
<script>

    // делаем первый шаг в грамотном решении проблемы

    // функция получения данных

    function getData() {

        // моделируем запрос к API (на внешние ресурсы)

        setTimeout(() => {

            // определение данных

            user = {name:"John", login:"master"};

            // вызов функции getOut по месту (IIFE)

            (function outData() {

                html = `

                    <h2>Здравствуйте, вы вошли в приложение
```

```
                как:</h2>
                Имя: <b>${user.name}</b> <br>
                Логин: <b>${user.login}</b> <br>
            `;
            // вывод данных в браузер
            document.getElementById("data").innerHTML = html;
        }) ();
    }, 500);
};
// вызов функции получения данных
getData();
</script>
```

Работает прекрасно, а выглядит не очень, ненужный, громоздкий HTML- код вывода в секции поставщика данных. Вернем **функцию outData** в глобальную область видимости, а в качестве аргумента функции **getData** передадим **описание** (код) **функции outData**.

Таким образом, становится понятен сакральный смысл **обратного вызова** (**callback**), это некий код (код функции), который передается в качестве аргумента на вход другой функции для выполнения **в определенный момент времени**. Выполнение достигается вызовом функции-аргумента с использованием круглых скобок - **callback**().

example_10. Создаем коллбэк для обработки внешних данных

```
<script>
    // передаем код другой функции в качестве аргумента
    // такой аргумент и называется коллбэком
    function getData(collback) {
        // моделируем запрос
```

```
    setTimeout(() => {
        // получаем данные, в реальном приложении это
        результат внешнего запроса
        user = {name:"John", login:"master"};
        // аргумент функции - функция коллбэк
        // добавлая скобки коду коллбэка - вызываем функцию
        collback();
    }, 500);
};
// функция вывода данных
function outData() {
    // интерполируем данные в строку
    html = `
        <h2>Здравствуйте, вы вошли в приложение как:</h2>
        Имя: <b>${user.name}</b> <br>
        Логин: <b>${user.login}</b> <br>
    `;
    // вывод данных в браузер
    document.getElementById("data").innerHTML = html;
};
// вызов функции получения данных
// вызываем функцию getData
// функция outData - аргумент-коллбэк
getData(outData);
</script>
```

Если написанное не очень понятно, вернитесь к коду **example_5** и попробуйте выполнить примеры еще раз.



Варианты callback

Callback - это не только работа с API, это гораздо больше. Возможны и другие варианты использования.

В следующих демонстрационных примерах представлены функции, которые **получают на вход итерируемый набор**, и **для каждого элемента набора вызывают функцию** обратного вызова **callback**.

В примере **example_11** функция и коллбэк для простого массива строк.

example_11. Callback и массив строк

```
<script>
    // определяем функцию
    // 1 параметр - массив
    // 2 параметр - коллбэк (код функции)
    function fnArgArr(list, callback) {
        // инициируем результирующий массив
        let result = [];
        // выполняем перебор элементов входного массива
        for (let elem of list) {
            // для каждого элемента вызываем функцию-коллбэк
            // результат записываем в массив
            result.push( callback(elem) );
        }
        // возвращаем результирующий массив
        return result;
    }
    // исходный массив
```

```
    arr = ["PHP", "JavaScript", "jQuery", "Bootstrap", "SQL"];
    // функция коллбэк
    let callback = elem => {
        return "#" + elem.toLowerCase();
    }
    // получаем преобразованный массив
    let tags = fnArgArr(arr, callback);
    let line = tags.join(" ");
    // вывод преобразованного массива в документ
    document.write(line);
    // вывод преобразованного массива в консоль
    console.log(tags);
</script>
```

В примере **example_12** функция и коллбэк для коллекции HTML-элементов.

example_12. Callback и коллекция HTML-элементов

```
<script>
    // определяем функцию
    // 1 параметр - итерируемый набор - коллекция элементов
    // 2 параметр - коллбэк (код функции)
    function fnArgNode(collection, callback) {
        let result = [];
        // выполняем перебор элементов коллекции
        for (let elem of collection) {
            // для каждого элемента вызываем функцию-коллбэк
            // результат записываем в массив
```

```
            result.push(callback(elem) );
        }
        // возвращаем массив
        return result;
    }
    // коллекция элементов p
    nodes = document.querySelectorAll("div p");
    // функция коллбэк
    let callback = elem => {
        // вывод в браузер
        document.write( "<h3>" + elem.innerText + "</h3>");
        // возврат в вызывающую функцию
        return  elem.innerText;
    }
    console.log(fnArgNode(nodes, callback));
</script>
```

Функция и коллбэк для внешнего массива данных.

example_13. Callback и объект

```
<script>
    // определяем функцию
    // 1 параметр - массив объектов
    // 2 параметр - коллбэк (код функции)
    function fnArgData(data, callback) {
        // моделируем запрос к API (на внешние ресурсы)
        setTimeout(() => {
```

```
            let line;
            // цикл поиска объекта по id
            for(item of data) {
                line += callback(item);
            }
            document.getElementById("app").innerHTML = line;;
            }, 1000);
        }
        // функция-коллбэк
        let callback = (elem) => {
            return `
            <b><a href='?id=${elem.id}'>${elem.name}</a></b><br>
            Бренд: <b>${elem.brand}</b><br>
            Тип двигателя: <b>${elem.dvs_type}</b><br>
            Мошность: <b>${elem.power_hp}</b> (л.с.)<br>
            Количество цилиндров:
            <b>${elem.cylinders_num}</b><br>
            Макс. скорость: <b>${elem.max_speed}</b><br>
            <img src='${elem.image}'><br>
        `;
    };
    fnArgData(cars, callback);
</script>
```

Таким образом, имея код функции и аргумент **callback** можно **экспериментировать** в решении поставленной задачи.



Закрепляем пройденный материал. Больше практики рассмотрим в **самостоятельных работах**:

- Преобразование цепочки вызовов в вызов функции с аргументом callback.
- Создание функций callback.
- Ревьюирование кода функций callback.
- Определение функции callback в зависимости от выбора пользователя.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

